Document number: P3238R0
Audience: SG21

Ville Voutilainen
2024-04-15

An alternate proposal for naming contract semantics

Abstract

The current naming of contract semantics more or less follows the form

_____ the trueness of a predicate

and we use the words 'ignore', 'enforce', 'observe', and ostensibly, 'quick_enforce' for the blank.

This proposal suggests that we should use the form

A contract violation is treated as _____

and that we use the words 'ignored', 'enforced', 'observed' and 'erroneous' for the blank.

So, this paper is proposing two things

The more significant part proposed here is that a contract with the semantic where a contract violation causes instant termination without calling a violation handler is simply treated as Erroneous Behavior, with the "fallback" defined behavior being calling the violation handler.

As far as I can see, that's perfectly reasonable. The semantic treats contract violations as catastrophic failures, so catastrophic that there is no user-defined code or any other hooks run on a violation, the program is just abruptly killed. For such a situation, it makes perfect sense to me to also allow the implementation to reject a program that runs into such a situation, as is already otherwise allowed for EB. EB allows implementation-defined diagnostics, so we can just make it have the defined behavior of calling the violation handler, and then an implementation can avoid doing that and produce a custom diagnostic instead.

So, the second part is then the renaming. I can't come up with a good action-verb for "be erroneous", like we have for 'ignore', 'enforce', and 'observe', and there's no good one for 'quick_enforce'. So I propose to just move that cheese and use a different approach, where the semantic 'erroneous' is in the same grammatical position as the others that are past tense verbs.

A look at a hypothetical future

We might eventually gain the ability to decide a contract semantic in source code. My preference for it isn't strong, but I find

```
void f(int x) pre enforced(x >= 0);
```

to make more sense than

```
void f(int x) pre enforce(x >= 0);
```

and I find

```
void f(int x) pre erroneous(x >= 0);
```

to make more sense than

```
void f(int x) pre quick_enforce(x >= 0);
```

Or perhaps

```
void f(int x) pre<erroneous>(x >= 0);
```

as opposed to

```
void f(int x) pre<quick_enforce>(x >= 0);
```